Bitácora de Registro Diario

Analizador Sintáctico: Implementación del algoritmo de la Colección canónica – Asignación 4

Equipo: Arenas Rojas Ivri Nitzani · Bautista Jiménez Israel · Hernández Torija Itzel Amairani · Juárez Jiménez Yarelli Sinai · López López Xochitl Nerina · Quiroz Martínez Maybelin

Periodo: 12 – 16 de octubre de 2026

Inicio de la Asignación 4

(12 y 13 de octubre de 2026)

Integrante	12/10/2026	13/10/2026
Arenas Rojas Ivri Nitzani	Coordinó la reunión de arranque de la Asignación 4, presentó los requerimientos del algoritmo de la colección canónica (entrada: gramática aumentada; salida: estados del AFD en una ventana) y repartió los módulos entre los integrantes	Creó el paquete `persistencia/analizadorSintacticoLR/ccanonica`, como en la estructura del proyecto del curso, y la carpeta `gramaticasAumentadas`
Bautista Jiménez Israel	Revisó la ventana de Primeros y Siguientes de la Asignación 3 y definió la distribución de la nueva ventana: gramática de entrada y gramática aumentada a la izquierda, y colección canónica a la derecha	Definió cómo distinguir en la salida los estados nuevos (rosa), los Ir_a que llevan a un estado existente (gris) y la aceptación (verde), reutilizando los colores e iconos de `estilos.py`
Hernández Torija Itzel Amairani	Analizó el formato de la gramática aumentada del enunciado: la primera regla `E'→E$` aparece aunque `E'` y `$` no estén declarados en el encabezado	Diseñó la función `aumentar()`: deja $S' \to S\,\$$ como producción 0 o la agrega si falta, y usa S'' cuando S' ya existe, como E' en la Gramática 6
Juárez Jiménez Yarelli Sinai	Estudió los elementos LR(0) y los algoritmos de cerradura e Ir_a del Tema 3, y calculó a mano I_0 de la gramática de expresiones E, T y F	Diseñó la representación de cada elemento $[A \to \alpha \cdot \beta]$ como la tupla `(p, k)` (producción y posición del punto) y el método `simbolo_despues_punto()`
López López Xochitl Nerina	Estudió el algoritmo de la colección canónica y el ejemplo de salida de la asignación: `cerradura({…})`, `Ir_a(Ii,X)`, `= Ij` para estados repetidos y `Aceptación` con `$`	Diseñó la clase `ColeccionCanonica`: los estados como listas (para conservar el orden de la salida), su comparación como conjuntos y el registro de pasos de la corrida
Quiroz Martínez Maybelin	Recopiló las seis gramáticas de la asignación y el ejemplo de salida de la Gramática 1 (estados I_0 a I_{11}) para compararlos después con el programa	Creó los archivos `g1` a `g6` aumentados en `gramaticasAumentadas` y calculó a mano los estados esperados de las Gramáticas 2 y 5

Desarrollo y entrega

(14 al 16 de octubre de 2026)

Integrante	14/10/2026	15/10/2026	16/10/2026
Arenas Rojas Ivri Nitzani	Agregó al submenú *Analizador sintáctico* la opción *Algoritmo Colección Canónica* y programó `coleccion_canonica()` en `VentanaPrincipal`, que abre la ventana hija como diálogo modal	Integró los módulos de todos, verificó la ejecución completa con `python3 -m compi.presentacion.compilador` y redactó la introducción, el análisis y la arquitectura por capas del informe	Dirigió la reunión final, ensambló el informe y la distribución de responsabilidades, empaquetó el proyecto en `Arquitectura_software_CC.zip` y entregó la Asignación 4
Bautista Jiménez Israel	Programó `VentanaColeccionCanonica`: el formulario con los botones Abrir, Analizar, Limpiar y Guardar resultados, los tres paneles de texto y `abrir_archivo()` con `filedialog` sobre la carpeta `gramaticasAumentadas`	Conectó `analizar()` con el lector y el cálculo, programó `guardar_resultados()` y `limpiar()`, tomó las capturas de las seis gramáticas y documentó la integración con la interfaz gráfica en el informe	Participó en la revisión final y validó la interfaz con las pruebas de las seis gramáticas aumentadas
Hernández Torija Itzel Amairani	Programó `aumentar()` y agregó a `LectorGramatica` el reconocimiento de la producción aumentada: su lado izquierdo pasa a ser el símbolo inicial y `$` se agrega a los terminales	Dibujó el diagrama de clases y documentó en el informe el formato de la gramática aumentada y el manejo de errores	Participó en la revisión final del proyecto y validó la versión entregable del sistema
Juárez Jiménez Yarelli Sinai	Programó `cerradura()`, que recorre la lista de elementos mientras crece, y `nucleo_ir_a()` e `ir_a()`, que avanzan el punto sobre un símbolo y aplican la cerradura	Comprobó que la salida de la Gramática 1 coincide con el ejemplo de la asignación (los mismos 12 estados en el mismo orden) y documentó en el informe los algoritmos y la tabla paso a paso	Participó en la revisión final del proyecto y validó la versión entregable del sistema
López López Xochitl Nerina	Programó `calcular()`, que detecta los estados repetidos y la aceptación, `simbolos_transicion()` y el formato de la salida (`formatear_elemento()`, `lineas()` y `reporte()`)	Revisó el número de estados y de Ir_a de cada gramática y documentó la salida del programa en las tablas de resultados del informe	Participó en la revisión final del proyecto y validó la versión entregable del sistema
Quiroz Martínez Maybelin	Calculó a mano los estados esperados de las Gramáticas 3, 4 y 6 y verificó que el lector acepte los seis archivos aumentados (`E'` y `$` sin declarar, ε y `V'`)	Ejecutó las pruebas de las seis gramáticas desde la interfaz (Abrir → Analizar), las comparó con el ejemplo y con el cálculo manual y registró los resultados en el informe	Revisó que el paquete entregable incluyera las gramáticas aumentadas y participó en la revisión final